WhisperChain Security Design Report

WhisperChain+ anonymous messaging platform | Final design assessment | November 11, 2026

1 Purpose and security objectives

WhisperChain+ lets members of a learning community send anonymous, encrypted compliments while providing controls against spam and abuse. Our implementation uses a Python command-line client, a FastAPI service, SQLite, and the cryptography library. Its main objectives are message confidentiality, separation of role permissions, one accepted message per sender credential per round, and moderation supported by privacy-preserving audit events. The core workflows are implemented, but the current prototype has unresolved security gaps. This report distinguishes implemented protections from the additional controls required for network deployment.

2 Architecture and trust boundaries

The client generates user keys, encrypts messages, and decrypts inboxes and moderation reports. The API approves accounts, validates credentials, routes encrypted envelopes, and records events. SQLite stores account records, public keys, encrypted messages, flags, and token-use state. Separate server key files support credential issuance and audit signing. User private keys remain on the client and are password-encrypted; local key and session files are created with restrictive permissions.

The client-to-service connection, the service-to-database connection, and recipient-to-moderator disclosure are distinct trust boundaries. Message encryption protects content stored by the service; it does not conceal every routing detail. The database retains recipient usernames and delivery state. The service sees a persistent pseudonym when a token is redeemed, and can observe request timing and network addresses. Anonymity therefore means avoiding a direct account-to-token association in issuance records, not protection against every traffic analyst or a fully compromised server.

3 Roles and access control

Role	Implemented permissions
Sender	Obtain one blind-signed credential after approval; submit a message using a valid, unused round token.
Recipient	Read only their own encrypted inbox; decrypt locally; flag only messages in that inbox.
Moderator	Retrieve encrypted flagged reports; decrypt locally; dismiss flags or freeze a pseudonym; read audit events.
Admin	Approve or revoke users, assign roles, open or close rounds, and read audit events. No message-reading endpoint is granted by the Admin role.

Authenticated routes check current account status and roles. Sender and Recipient roles may coexist, while assigning Admin and Moderator together is rejected. Object-level ownership checks protect inboxes and flagging. Public directory and token-verification keys require no login. Anonymous submission intentionally uses a credential instead of an account session. This preserves issuance privacy but complicates revocation, as discussed on page 3.

4 Anonymous credentials and round enforcement

After approval, a Sender creates an Ed25519 pseudonym key. The client blinds a hash of its public key with the server RSA public key. The authenticated issuance endpoint signs the blinded value and records only that this account received a credential. The client unblinds and verifies the signature. Issuance is limited to one credential per account for its lifetime; a new round does not require another authenticated issuance.

For each round, the pseudonym key signs the round identifier. Submission carries the public pseudonym, its credential, and that round signature without an account bearer token. The service verifies both signatures, rejects frozen pseudonyms, and uses a database uniqueness constraint on pseudonym hash plus round identifier to reject replay. Token consumption and message insertion share a transaction. This limits accepted messages per credential, but does not prevent account farming, credential sharing, or network-level floods. Stable pseudonyms also make activity linkable across rounds. The blind-signature construction is educational textbook RSA over SHA-256, not an independently reviewed implementation of a standardized blind-signature protocol.

5 Message encryption and secure delivery

At signup, the client generates a 3072-bit RSA encryption key pair and uploads only the public key. A Sender downloads the public directory, checks the recipient key against a locally pinned fingerprint, and creates an inner message containing text, recipient, round, pseudonym, credential, and an Ed25519 signature. The client pads this payload to 4096 bytes, encrypts it with a fresh AES-256-GCM key and random 12-byte nonce, and wraps the AES key with recipient RSA-OAEP using SHA-256. Authenticating the recipient name prevents an envelope from being moved successfully to another inbox.

The service checks envelope shape and stores ciphertext with a random message identifier, recipient username, and delivery flag. There is no sender field or message timestamp in the message table. Recipients fetch their own envelopes and decrypt locally; the client displays content only after checking the pseudonym credential and signature. Fixed payload size reduces message-length leakage. Fingerprint pinning detects later recipient-key changes, but first use still depends on an authentic directory. User-key compromise exposes content encrypted to that key; the protocol does not provide forward secrecy.

6 Moderation and audit accountability

Flagging re-encrypts the decrypted, signed inner message for every active moderator. The service stores those reports as ciphertext, and moderators decrypt and verify them locally. Moderators added after a report was created cannot decrypt that report. Freezing records a pseudonym hash that blocks its future tokens in every round. Alerts become audit events when open flags reach five or distinct recorded freezes in the preceding 24 hours reach three; the client dashboard displays these events.

Major events record UTC time, action, and role without account names or message text. SQLite triggers reject ordinary audit updates and deletes. Each entry incorporates the previous hash, and a system Ed25519 key signs its hash. These mechanisms expose many edits, but a privileged database operator can remove triggers, and a valid suffix can be deleted without an external checkpoint revealing the truncation. The system key authenticates log entries; admin and moderator actions are not individually signed, and role-only logs cannot identify which person performed an action.

7 Validation and observed limitations

The existing automated suite completed with 38 passing tests. It covers registration and approval, role restrictions, token replay and forgery rejection, encryption and decryption, envelope tampering, inbox ownership, flagging, freezing, and audit integrity. Two additional checks in a disposable test database exposed behavior outside that coverage: an account-revoked Sender still submitted with an existing credential (HTTP 201), and a Moderator froze an unrelated pseudonym by supplying it against a valid open flag (HTTP 200). These results demonstrate limits of the current enforcement rather than successful defenses.

Priority	Security gap and required response
High	Transport: the supplied client defaults to HTTP and the setup contains no TLS configuration. Require authenticated HTTPS outside loopback so passwords, bearer tokens, and directory responses are protected.
High	Revocation: existing credentials survive account revocation or Sender-role removal. Design anonymous credential expiry or revocation before claiming that current role eligibility is enforced at redemption.
High	Freeze integrity: the server trusts a moderator-supplied pseudonym. Require verifiable evidence binding the flag, submitted envelope, and freeze target; client-only verification can be bypassed.
High	Privileged abuse: role reassignment and new account approval let a compromised Admin weaken separation of duties. Introduce independent approval for sensitive role and key changes.
Medium	Availability and privacy: general rate limits, metadata-retention controls, independent audit checkpoints, and a documented key-recovery and rotation process are not implemented.

8 Design trade offs and completion criteria

Blind issuance is our advanced feature: it separates account approval from anonymous credential use, while still allowing one-use-per-round enforcement. This adds client key management, prevents simple account-based revocation, and leaves a stable pseudonym for cross-round abuse control. Replacing it with an ordinary authenticated send request would simplify revocation but expose account-to-message linkage. A revised design must make that trade-off explicit rather than silently weakening anonymity.

The RBAC matrix must be reconciled with the code. It restricts directory browsing although the endpoint is public, and promises key replacement or rotation, configurable alert thresholds, unfreezing, and second-person approval that are absent. The original draft also mentions browser cryptography, general rate limiting, and individual privileged-action signatures; the implemented client is command-line based and these latter controls are not present. Before network deployment, resolve the high-priority findings, add regression tests for them, update the matrix, and demonstrate the complete send, decrypt, flag, freeze, and audit flow with analyzed audit samples.

Evidence and implementation references

Assignment: WhisperChain+ Anonymous Messaging with Role-Based Control and Accountability, including the final submission checklist. Project evidence: README.md; Week 1 Deliverables/Design Doc and Threat Model.md; RBAC Matrix - Sheet1.pdf. Implementation: client.py and whisper_crypto.py; app/main.py, auth.py, tokens.py, messages.py, moderation.py, db.py, audit.py, and blind.py. Validation: tests/ plus the two isolated checks described above. The companion Final Threat Model provides the detailed risk register and verification criteria.